

Lab Work Assignment 3: Data Version Control (DVC) Pipeline Automation

Course: ML Operations

Objective:

The goal of this lab is to design and implement a data version control system for automating machine learning workflows. Students are expected to set up a DVC pipeline, manage data dependencies, and demonstrate how the pipeline ensures reproducibility in the training and evaluation process.

Resources:

- [Example DVC project on github](#)
- [DVC Documentation](#)
- [DVC GitHub](#)
- [PyTorch Tutorial for Model Training](#)

Submission Deadline:

Submit your completed project and lab report by 08.10.2024.

Part 1: Project Setup with DVC

DVC Initialization:

Task: Install DVC and initialize a project with DVC. Ensure that your project includes a `.gitignore` file that excludes unnecessary files from version control.

Data Tracking:

Task: Add your dataset files to DVC tracking. Use DVC to track both the raw data files and a dataset registry table, where relevant information about the dataset versions is stored.

Remote Setup (optional):

Task: Set up your local directory as a DVC remote for storing versioned data. Demonstrate how you can push and pull data to and from this remote location.

Part 2: Pipeline Creation with DVC

Convert Scripts to DVC Pipeline Stages:

Task: Convert your existing scripts (e.g., data download, model training, and evaluation) into distinct DVC pipeline stages. Each stage should have clearly defined inputs, outputs, and dependencies managed by DVC.

Pipeline Requirements:

- The **data download** stage should fetch raw data and save it to a tracked location.
- The **training** stage should depend on the downloaded data and produce model artifacts.

- The **evaluation** stage should depend on the trained model and report performance metrics.

Task: Demonstrate the use of `dvc repro` to rerun the entire workflow when there are changes in data or code.

Part 3: Parameterization and Configuration

Parameter Management:

Task: Use `params.yaml` to manage the key parameters for your pipeline stages, such as model hyperparameters, dataset configurations, and evaluation thresholds. Ensure that your pipeline can dynamically adjust to changes in the `params.yaml` file.

Part 4: Code Management and Best Practices

Configuration and Code Management:

Task: Ensure all pipeline configuration parameters are stored in `params.yaml`. Keep your code and DVC files organized for reproducibility.

Version Control:

Task: Use GitHub to manage your project. Commit your DVC files and versioned dataset changes, and push them to the repository. Demonstrate how you can track code and data changes together.

Logging:

Task: Use Python's logging module to log important events, including pipeline execution and the completion of each DVC stage.

Part 5: Lab Report

Content:

- **Introduction:**
Explain the importance of data version control in machine learning workflows. Introduce DVC and describe how you applied it to automate and version your data and pipeline.
- **Pipeline Description:**
Describe the DVC pipeline you implemented, including the stages (data download, training, and evaluation), their dependencies, and how `dvc repro` ensures reproducibility.
- **Parameterization:**
Present the key parameters stored in `params.yaml` and explain how they affect different stages of the pipeline.

- **Reflection:**
Reflect on the benefits and challenges of using DVC in your project. Suggest any improvements that could be made with additional resources or time.
 - **Code Submission:**
Submit your project repository on GitHub, ensuring all code, DVC files, and configurations are well-organized. Include documentation and a clear commit history.
-

Grading Criteria:

1. **Parameter Management:** Correct use of `params.yaml` for managing pipeline parameters.
2. **DVC Pipeline:** Effective implementation of a DVC pipeline with clear input-output dependencies across stages.
3. **Version Control:** Proper integration of Git and DVC, including committing code and data changes with clear commit messages.